



Progress Report 4

BC Personal Health Management Platform

An AI-Powered Healthcare Navigation and Analytics System

CSIS 4495 - 002: Applied Research Project
Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: March 16, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Prepared By:

<u>Name</u>	<u>Student ID</u>	<u>Email Address</u>
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Work Date/Hours Log	2
2. Summary Description of Work Done.....	5
2.1 Sprint 3D: Health Education Library	5
Knowledge Base Architecture (data/health_articles.json)	5
Search Functionality	5
Browse by Category	6
Popular Topics and All Articles Sections.....	7
Official BC Health Resources.....	7
2.2 Sprint 4A: Synthetic Health Assessment Dataset.....	8
Three-Tier Patient Model	9
Realistic Patterns.....	9
2.3 Sprint 4B: System Analytics Dashboard	10
Page Structure and KPI Cards	10
Clinical Severity Colour-Coded Symptom Chart	10
Urgency Distribution and BC Health Authority Charts.....	11
Assessment Trends and Seasonal Patterns	11
K-Means Unsupervised Clustering — Patient Segmentation.....	12
2.4 Issues Encountered and Solutions	14
3. Repo Check-in of Implementation Completed.....	15
4. Revised Timeline and Next Steps.....	18
4.1 What's Next: Sprint 5 and Sprint 6.....	19
4.2 Dual Machine Learning Architecture — Academic Framing	19
4.3 Optional Feature: Community Pulse Dashboard (Time-Permitting)	19
5. AI Use Section.....	21
5.1 AI Tools Usage Summary	21
5.2 Table of AI Tools and Specific Use.....	21
5.3 Statement of Originality	22
Appendix A: AI Prompt History.....	24
Prompt Session 1: JSON Knowledge Base Design for Health Library	24
Prompt Session 2: CSS :has() Selector Pattern for Streamlit Widget Styling.....	24
Prompt Session 3: Synthetic Data Generation with Stratified Patient Tiers	25
Prompt Session 4: K-Means Feature Engineering for Healthcare Patient Segmentation	26

1. Work Date/Hours Log

Note: Logs are entered as work is completed with granular entries. This reporting period covers Sprint 3D (Health Education Library), Sprint 4A (Synthetic Data Generation), and Sprint 4B (System Analytics Dashboard) from March 10–16, 2026.

Date	Hours	Description of Work Done
2026-03-11	1.00	Sprint 3D Research: Reviewed HealthLinkBC content categories and identified 8 topic areas for the Health Library (Pain & Injury, Cold & Flu, Medications, Prevention & Wellness, Mental Health, Heart Health, Child Health, Emergency Care)
2026-03-11	0.75	Sprint 3D Research: Designed JSON schema for health_articles.json — identified required fields: id, title, category, summary, content, tags, read_time, source_url, banner_color; mapped 18 article topics across 8 categories
2026-03-11	1.25	Sprint 3D: Drafted prompt for Claude Code to generate health_articles.json with 18 BC-specific health articles and 5_Health_Library.py with full page structure matching mockup; reviewed and committed first draft
2026-03-11	1.00	Sprint 3D: Tested initial Health Library page — identified and documented UI bugs: duplicate category grid rendering, search section layout issues, incorrect section header styling; planned targeted fix approach
2026-03-13	0.75	Sprint 3D Fix: Corrected Health Library UI — removed duplicate st.button() row above category icon cards, replaced teal section_header() on Browse by Category with plain st.markdown(), redesigned search card layout
2026-03-13	1.00	Sprint 3D Fix: Resolved ghost input box appearing between page subtitle and search card — root cause identified as orphaned HTML div wrappers auto-closing before Streamlit widgets rendered; replaced with CSS :has() marker injection pattern
2026-03-13	1.25	Sprint 3D: Implemented search logic — keyword filtering against article title, category, tags, and summary fields using session_state (hl_search_query); results grid with 3-column card layout, Clear Search button, no-results fallback message

Date	Hours	Description of Work Done
2026-03-13	0.75	Sprint 3D: Wired Browse by Category Select buttons to filter articles — hl_selected_category session state, 3-column filtered results grid, mutual exclusivity with search filter (selecting category clears search and vice versa); tested all 8 categories
2026-03-15	0.75	Sprint 3D: Verified Popular Topics section — 4 coloured banner cards (blue/green/purple/red), expandable full article content, Visit Source external links; confirmed all expanders and links functional
2026-03-15	0.50	Sprint 3D: Implemented UX improvement — wrapped All Articles section in collapsed st.expander() to reduce cognitive overload from 18 article cards; applied teal CSS :has() styling to match platform section header design system
2026-03-15	0.75	Sprint 3D: Standardized font weight across all teal section headers (700 weight, 16px, 0.01em letter-spacing) for visual consistency; confirmed Official BC Health Resources section (HealthLinkBC, BCCDC, BC Health Services cards) rendering correctly
2026-03-15	0.50	Sprint 3D: Final review and testing of complete Health Library — search, category filter, popular topics, all articles expander, BC health resources, footer disclaimer; committed all Sprint 3D work to GitHub
2026-03-16	1.00	Sprint 4A Research: Designed synthetic data model for System Analytics — defined 3 patient engagement tiers (Healthy/Occasional, Moderate Users, High-Frequency/Chronic) with realistic assessment frequency and urgency distributions; drafted data generation requirements
2026-03-16	1.25	Sprint 4A: Developed generate_synthetic_data.py — 200 patients across 3 tiers, 5,000 assessment records with tier-weighted urgency distributions, flu season patterns (Nov–Feb spike), chronic symptom elevation for Tier 3; post-generation Emergency cap at ≤5%
2026-03-16	0.50	Sprint 4A: Validated synthetic_assessments.csv output — confirmed 5,000 records, Emergency at exactly 5.0% (250 records), age distributions match tier specifications, top symptoms reflect chronic condition elevation for Tier 3 patients; committed to GitHub

Date	Hours	Description of Work Done
2026-03-16	1.50	Sprint 4B: Built System Analytics Dashboard page (6_System_Analytics.py) — page header with Export Report placeholder and date range filter dropdown (All Time default), CSV data loading with os.path relative resolution, @st.cache_data, 4 KPI cards with teal headers (Total Users, Active This Week, Total Assessments, Avg Assessment Time)
2026-03-16	1.75	Sprint 4B: Added 4 Plotly charts — horizontal bar chart (top 10 symptoms with clinical severity colour coding: red=Serious/Chronic, amber=Infectious, blue=General), Urgency Distribution donut, Usage by BC Health Authority donut (5 health authority regions mapped from cities), Assessment Trends line chart with numpy.polyfit orange trendline
2026-03-16	1.50	Sprint 4B: Implemented K-Means unsupervised clustering — per-patient aggregation (age, assessment count, urgent ratio, chronic ratio), StandardScaler normalization, KMeans(n_clusters=3), automatic cluster naming by combined severity score; scatter plot visualization with bubble size = age; 3 cluster summary cards
2026-03-16	0.50	Sprint 4B: Final verification and testing — date range filter wiring confirmed across all KPI cards and charts, K-Means cluster separation validated (Chronic/Elderly avg age 73, High-Risk avg age 47, Healthy/Occasional avg age 29); committed all Sprint 4A and 4B work to GitHub

TOTAL 17.25 hours logged for Progress Report 4 (Mar 10–16, 2026)

2. Summary Description of Work Done

During this reporting period (March 10 - 16, 2026), three significant workstreams were finalized: the complete implementation and troubleshooting of the **Health Education Library** (Sprint 3D), creation of a **validated synthetic dataset for health assessment** (Sprint 4A), and development of **the System Analytics Dashboard** with user-friendly Plotly charts and K-Means unsupervised machine learning clustering (Sprint 4B). These sprints complete all leftover Sprint 3 features and provide the primary analytics functionality intended for Sprint 4, making the platform to near-feature-complete just before the final deployment sprint.

2.1 Sprint 3D: Health Education Library

The Health Education Library (pages/5_Health_Library.py) functions as a standalone portal of BC health and offers to the users in the form of a static JSON knowledge base. It provides a keyword search, category-based browsing, expandable article cards, and direct links to official BC health resources.

Knowledge Base Architecture (data/health_articles.json)

The library is powered by a pre-authored JSON knowledge base containing 18 health articles across 8 clinically relevant categories. Each article record contains the following fields:

- title, category, summary, content — article metadata and full body text
- tags — list of keywords for search matching
- read_time — estimated reading duration
- source_url — corresponding HealthLinkBC page URL for source verification
- banner_color — used for Popular Topics coloured banner display

Article content was generated using AI assistance as placeholder health education summaries structured around HealthLinkBC topic areas. The articles however contain a source URL that points to the official HealthLinkBC page related to the article so that users can verify and read the authoritative source directly. Therefore, in this arrangement, the JSON content acts as a carefully selected summary layer rather than a substitute for professional health information.

Search Functionality

The search section contains that Find Health Information card that is centered and is equipped with a keyword search bar and also popular search chips (Headache, Fever, Back Pain, Diabetes, High Blood Pressure). When a user enters a word or phrase and hits the Search button, the system does a case-insensitive keyword matching against the article title category tags, and summary fields fetched from the JSON knowledge base. Below are the results displayed in a 3-column card grid with each card

showing category badge title summary, and read time. Besides these, each card has a Read More expander that when clicked exposes the entire article content and a Visit Source link leading to the official HealthLinkBC page.

If there are no relevant articles for the search query, a no-results fallback message (st.info) will be displayed informing users to browse by category instead. The Clear Search button when clicked will clear the search condition and revert to the default page display.

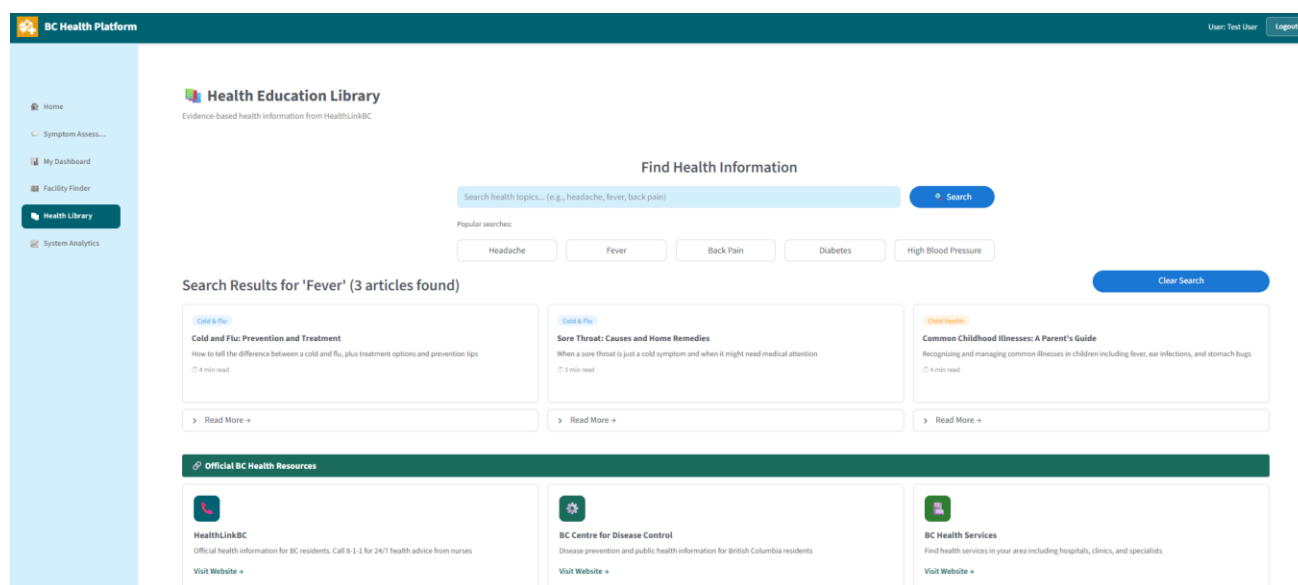


Fig 2.1a – Health Library search results showing article cards with category badges and Read More expanders

Browse by Category

Eight category cards are arranged on a 2x4 grid with each card containing a circular colored icon, bold category name, and short description. Selecting a category by clicking “Select” filters the articles display to reveal only the articles from this selected category. The heading indicates the number of the articles (e. g. Articles in 'Heart Health' (2 articles found)). A Clear Filter button removes the selected category. Category filtering and keyword search cannot be combined - using one will deactivate the other in order to prevent conflicting filter states.

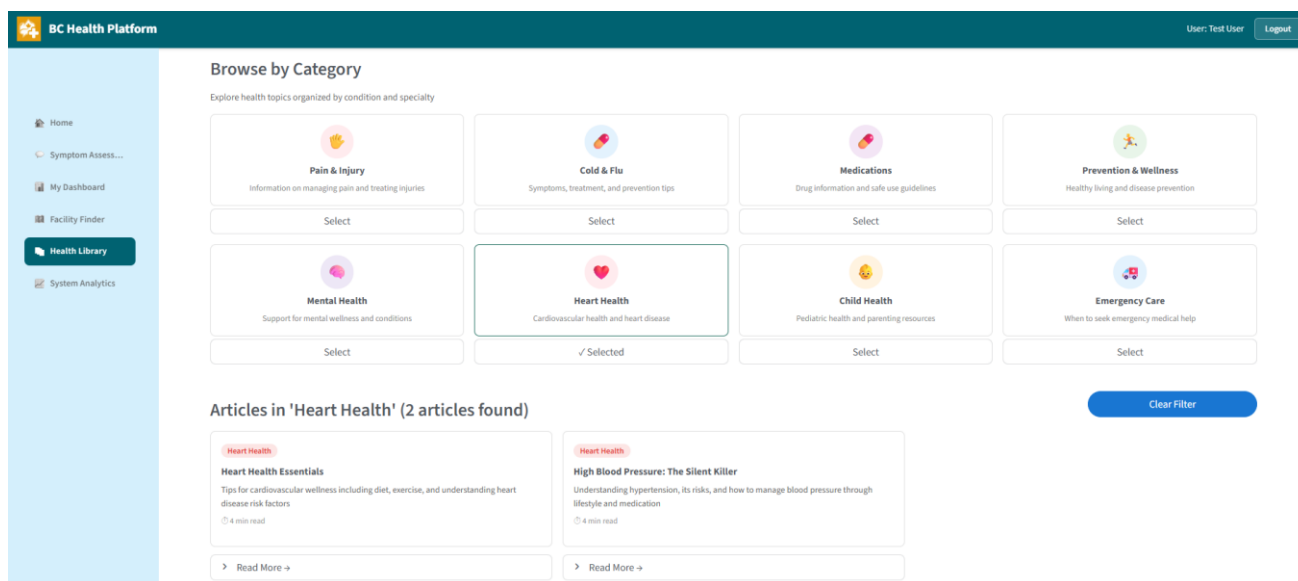


Fig 2.1b – Browse by Category with Mental Health filter active showing filtered article cards

Popular Topics and All Articles Sections

The Popular Topics area shows 4 top articles as coloured banner cards (blue green purple, red) arranged in a 2x2 grid. Every card shows the article icon title summary, and a collapsible “Read More” portion with the whole article content and a Visit Source link to HealthLinkBC.

As a default, the All Articles part is enclosed within a collapsed `st.expander()`, which helps to minimize the visual strain caused by showing all 18 cards simultaneously. Such a design choice makes users lean more towards Search or Browse by Category for the targeted navigation. The expander with the teal (`#1a6b5c`) CSS style is applying the `:has()` marker injection pattern to conform to the platform-wide section header design system.

Official BC Health Resources

Three resource cards have direct links to the official BC health portals: HealthLinkBC (8-1-1, 24/7 nurse advice), BC Centre for Disease Control (public health information), and BC Health Services (facility and specialist finder). Each card also has the Visit Website external link.

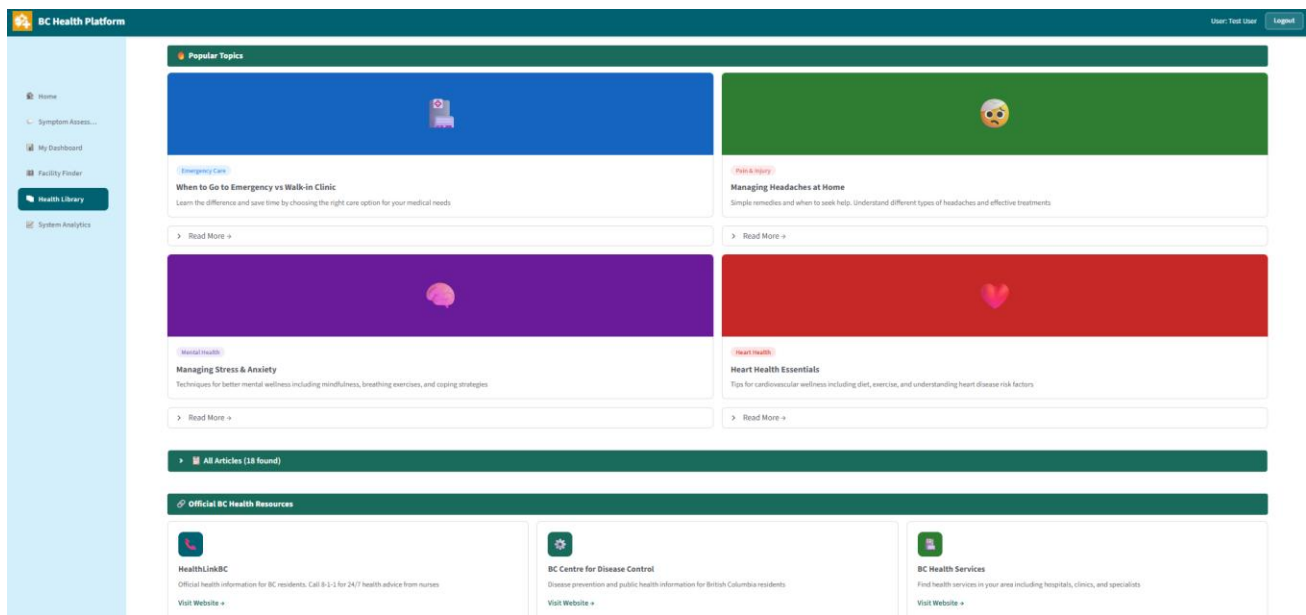


Fig 2.1c – Health Library lower sections showing Popular Topics banners, *collapsed All Articles expander*, and BC Health Resources cards

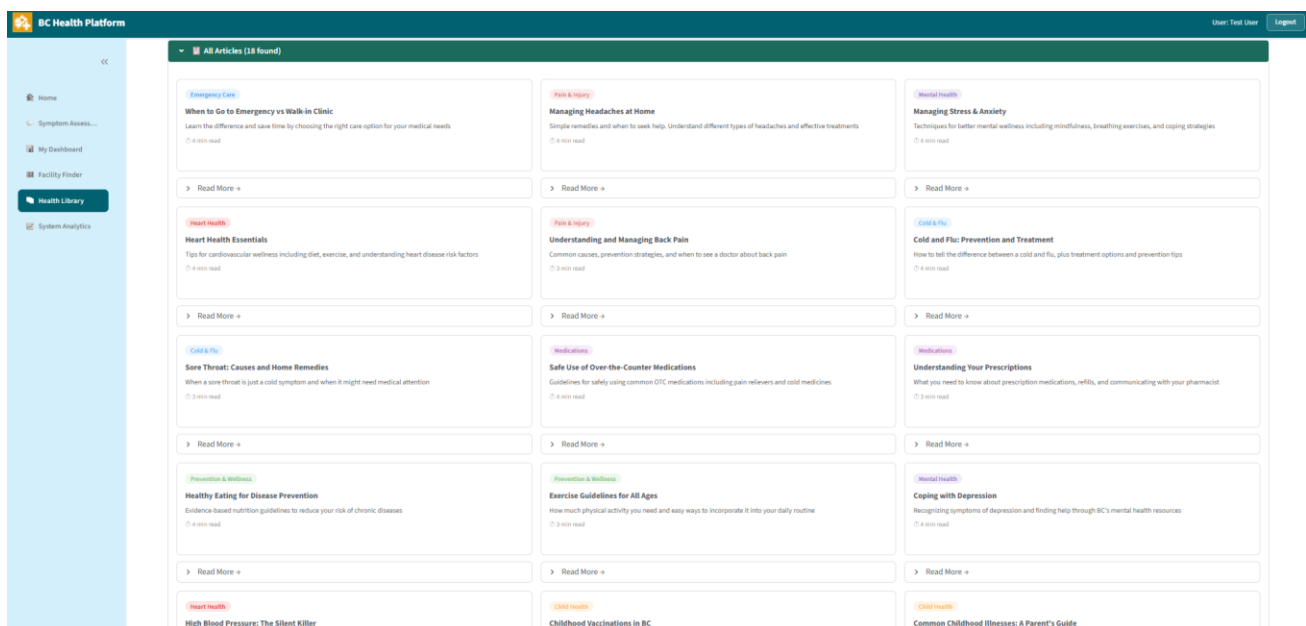


Fig 2.1d –Health Library *All Articles* section expanded showing all 18 article cards across 8 categories

2.2 Sprint 4A: Synthetic Health Assessment Dataset

Before the creation of the System Analytics Dashboard, a realistic synthetic data set was generated that mirrors platform-scale analytics data. The data set is representative of the 200 active users of the BC platform at three different levels of engagement. It effectively models the usage pattern of a digital health management tool where chronic condition patients play a major role in total assessment volume.

Three-Tier Patient Model

Patients were assigned to one of three engagement tiers at creation, each with distinct demographic and clinical profiles:

Tier	Patients	Age Range	Assessments/Year	Urgency Profile
Tier 1 — Healthy/Occasional	80 (40%)	18–45	3–8	70% Routine / 2% Emergency
Tier 2 — Moderate Users	80 (40%)	30–65	15–25	50% Routine / 5% Emergency
Tier 3 — Chronic/High- Frequency	40 (20%)	50–85	50–80	30% Routine / 7% Emergency*

*Post-generation Emergency cap: a validation check was applied after generation to ensure total Emergency records never exceeded 5% of all 5,000 records, with excess records downgraded to Urgent. Final Emergency count: exactly 250 records (5.0%).

Realistic Patterns

Three realistic patterns were embedded into the synthetic data to make it analytically meaningful:

- Flu season spike (Nov–Feb): 40% higher frequency of cold/flu symptoms (fever, cough, sore throat, runny nose) during winter months, matching real BC seasonal health patterns
- Tier 3 chronic symptom elevation: chest pain, shortness of breath, high blood pressure, diabetes symptoms, and joint pain appear at higher proportions for Tier 3 patients
- Weekday bias: assessment dates are slightly more frequent on weekdays than weekends, reflecting realistic health platform usage patterns

I used random seed 42 to make sure the results are reproducible. The dataset was examined with a printed summary showing the number of records, urgency distribution, tier breakdown, top symptoms, and date range to Sprint 4B.

I would also like to point out that the synthetic dataset (synthetic_assessments.csv) is deliberately kept separate from the live SQLite database (health_platform.db) that stores real authenticated user assessments. The CSV offers a mock population dataset for demonstrating analytics. In a real-world setting, the System Analytics Dashboard would be linked to the live database that has aggregated data as the user base grows.

2.3 Sprint 4B: System Analytics Dashboard

The System Analytics Dashboard (pages/6_System_Analytics.py) provides platform-wide health analytics derived from the synthetic assessment dataset. It combines KPI metrics, interactive Plotly visualizations, and K-Means unsupervised machine learning to deliver population-level health insights.

Page Structure and KPI Cards

The page header consists of the dashboard title subtitle, an Export Report button stub (st.toast) and a date range dropdown (Last 7 Days / Last 30 Days / Last 90 Days / All Time, by default showing All Time). The date filter affects all metrics and charts throughout the entire page.

Four KPI cards are displayed in a st.columns(4) layout, each with a teal (#1a6b5c) header bar matching the platform-wide design system:

- Total Users: count of unique patients in the filtered dataset
- Active This Week: unique patients with assessments in the last 7 days of the dataset
- Total Assessments: total record count in the filtered dataset
- Avg Assessment Time: mean assessment_duration_seconds ÷ 60, formatted as X.X min

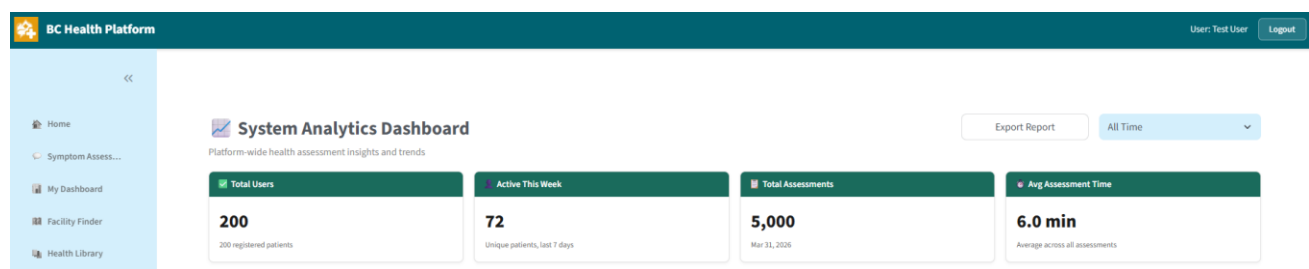


Fig 2.3a – System Analytics Dashboard KPI cards with teal headers matching platform design system

Clinical Severity Colour-Coded Symptom Chart

The Top Symptoms chart is a horizontal Plotly bar chart displaying the top 10 symptoms by assessment count. Bars are color-coded by clinical severity to communicate medical meaning rather than arbitrary ranking:

- Red (#dc3545) — Serious/Chronic: chest pain, shortness of breath, high blood pressure, diabetes symptoms
- Amber (#ffc107) — Infectious: fever, runny nose, cough, sore throat
- Blue (#1976D2) — General: fatigue, nausea, muscle soreness, joint pain, and other manageable symptoms

A color legend annotation (Serious/Chronic, Infectious, General) is displayed below the section header. This clinical severity classification follows triage principles where color signals urgency level rather than just frequency rank.

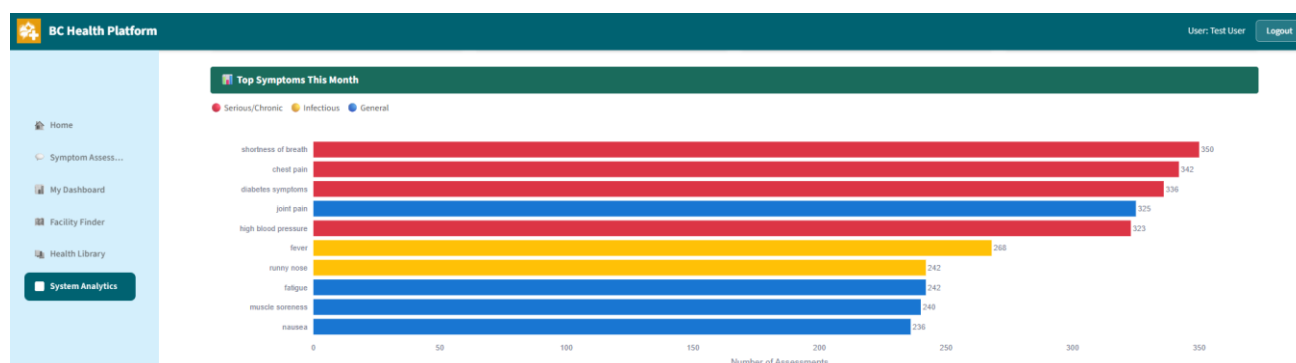


Fig 2.3b – Top Symptoms bar chart with clinical severity colour coding and legend annotation

Urgency Distribution and BC Health Authority Charts

Two donut charts are shown side-by-side with st. columns(2). The Urgency Distribution donut uses clinically meaningful colours: green (#28a745) for Routine, amber (#ffc107) for Moderate, orange (#fd7e14) for Urgent, and red (#dc3545) for Emergency. The Usage by Health Authority donut chart takes the 10 cities in BC from the dataset and maps them to the 5 official BC Health Authorities (Fraser Health, Vancouver Coastal Health, Interior Health, Island Health, Northern Health). The five Health Authorities are represented by five different shades of teal for visual distinction.

Assessment Trends and Seasonal Patterns

The Assessment Trends Over Time chart groups assessments by month using `strftime('%b %Y')` - the same format correction implemented in Sprint 3B - to prevent collapsing same-month data across years. The chart shows a teal area fill line chart with an orange dashed trendline derived via `numpy.polyfit`, matching the “My Personal Health” Dashboard trendline implementation. Data labels are only selectively shown at the maximum and minimum monthly data points to prevent visually overloading the chart (e. g. '440 Dec 2025' and '349 Feb 2026'). The December 2025 peak corresponds to the flu season spike which is part of the synthetic data generator hence the dataset mirrors actual seasonal health patterns.

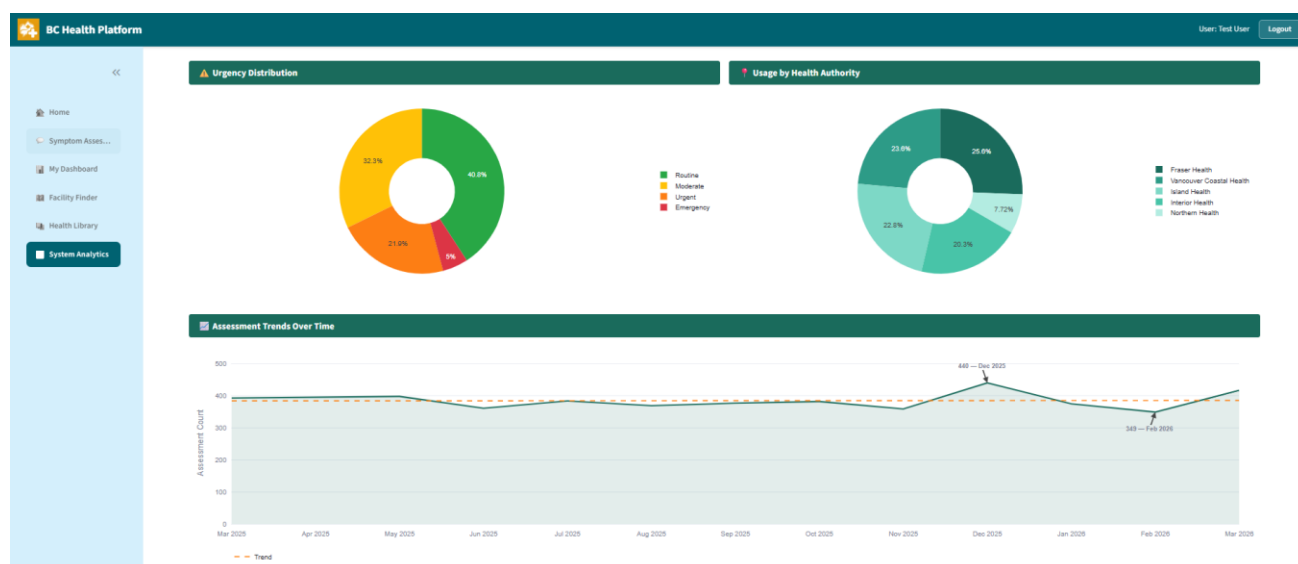


Fig 2.3c – Urgency Distribution, BC Health Authority donut charts and Assessment Trends

K-Means Unsupervised Clustering — Patient Segmentation

The Patient Cluster Analysis section demonstrates the platform's second machine learning technique. In contrast to the Symptom Assessment chatbot which is supervised (a Random Forest with labelled training data), the System Analytics Dashboard uses unsupervised K-Means clustering to find natural patient segments without any pre-defined labels. This dual ML approach is a mirror of the real-world clinical analytics scenario:

Type	Algorithm	Where Used	Purpose
Supervised	Random Forest	Symptom Assessment Chatbot	Predicts diagnosis from labelled training data
Unsupervised	K-Means Clustering	System Analytics	Discovers natural patient groupings without labels

For the clustering, each patient is aggregated into four features: mean age, assessment count, urgent ratio (proportion of Urgent+Emergency records), and chronic ratio (proportion of serious/chronic symptoms). Features are transformed to zero mean, unit variance using StandardScaler before fitting `KMeans(n_clusters=3, random_state=42)`. Clusters are named automatically by sorting on combined `chronic_ratio + urgent_ratio` score.

The three clusters are visualized on a scatter plot (X = assessment frequency, Y = urgent case ratio, bubble size = patient age) and pinpoint the following segments:

Cluster	Patients	Avg Age	Avg Assessments/Yr	Urgent Ratio
Chronic/Elderly Patients	40	73	70.5	33.6%
High-Risk Patients	86	47	18.8	22.3%
Healthy/Occasional Users	74	29	7.6	7.4%

There is a clinical rationale behind the separation of the clusters: elderly chronic patients (the average age of the group is 73) are the primary users of the platform and at the same time have the highest urgency ratios because their chronic conditions not only necessitate more frequent assessments but also are prone to cause more serious clinical problems. This type of segmentation can be a basis for tailoring targeted BC healthcare interventions, for instance, a proactive outreach to Chronic/Elderly patients who regularly display high urgency ratios.



Fig 2.3d – K-Means patient segmentation scatter plot with 3 cluster summary cards below

2.4 Issues Encountered and Solutions

Issue	Impact	Solution
Ghost input box rendering between page subtitle and search card (Health Library)	Empty white rectangle appearing above the Find Health Information section; caused by orphaned <code>st.markdown('<div>')</code> tags auto-closing before Streamlit widgets rendered inside them	Replaced raw HTML div wrappers with CSS <code>:has()</code> marker injection pattern — invisible <code></code> marker elements allow CSS to style Streamlit's own DOM containers without orphaned HTML
Popular search chips not clickable after converting to HTML spans	Chips rendered correctly as styled pills but could not update <code>session_state</code> for search filtering	Reverted chips from static HTML spans to <code>st.button()</code> elements styled via CSS to maintain both visual appearance and click interactivity for <code>session_state</code> updates
Tier 3 Emergency urgency potentially exceeding 5% global cap	Tier 3 patients at 7% Emergency × ~2,600 records would produce ~182 Emergency records from Tier 3 alone, potentially exceeding the 5% total cap when combined with other tiers	Added post-generation validation check in <code>generate_synthetic_data.py</code> : if total Emergency records exceed 250 (5% of 5,000), excess records are randomly downgraded to Urgent until the cap is exactly met
K-Means cluster naming unreliable if based on single-feature ranking	Clusters could be misnamed if one feature (e.g. <code>chronic_ratio</code>) ranked differently from <code>urgent_ratio</code> , resulting in mismatched clinical labels	Named clusters by ranking on combined <code>chronic_ratio</code> + <code>urgent_ratio</code> score, ensuring the most clinically serious cluster is always labelled Chronic/Elderly Patients regardless of individual feature variation

3. Repo Check-in of Implementation Completed

GitHub Repository: https://github.com/desporamon/W26_4495_S2_DesmondC

The following files were created or modified since Progress Report 3. All work is committed with descriptive messages and pushed to the main branch:

File / Folder	Description
Implementation/data/health_articles.json	New data file: 18 BC health articles across 8 categories (Pain & Injury, Cold & Flu, Medications, Prevention & Wellness, Mental Health, Heart Health, Child Health, Emergency Care); fields include title, category, summary, content, tags, read_time, source_url, banner_color
Implementation/src/pages/5_📖_Health_Library.py	New page: Health Education Library with keyword search (session_state filtering), Browse by Category (8 icon cards with article filter), Popular Topics (4 coloured banner cards with expanders), All Articles (collapsed expander), Official BC Health Resources section, footer disclaimer
Implementation/data/synthetic_assessments.csv	New data file: 5,000 synthetic health assessment records across 200 patients (3 engagement tiers); includes patient demographics, symptom, diagnosis, urgency_level, recommendation, assessment_duration_seconds, confidence_score; Emergency capped at 5.0%
Implementation/src/data/generate_synthetic_data.py	New script: generates synthetic_assessments.csv with 3-tier patient model, flu season patterns, chronic symptom elevation for Tier 3, post-generation Emergency cap validation, and printed summary; random_state=42 for reproducibility

File / Folder	Description
Implementation/src/pages/6_📄_System_Analytics.py	Rebuilt page: System Analytics Dashboard with date range filter, 4 teal-header KPI cards (data from CSV), 4 Plotly charts (symptom bar with clinical severity colours, urgency donut, BC Health Authority donut, assessment trends with orange trendline), K-Means clustering section with scatter plot and cluster summary cards

Git Commit History (this reporting period):

- Mar 10: "Uploaded chatbot theory learning notes from NLP – Prof Ivan's class"
- Mar 13: "Sprint 3D: First draft of Health Education Library – articles JSON + Streamlit page"
- Mar 13: "Sprint 3D: Add search logic – session state filtering, results grid, clear button, no-results fallback"
- Mar 13: "Sprint 3D: Remove ghost input box and implement centered narrow search card with blue background band"
- Mar 13: "Sprint 3D: Fix ghost box with CSS :has() marker injection – no more raw HTML div wrappers"
- Mar 15: "Sprint 3D: Fix Health Library UI – search card layout, category grid duplication, section header styling"
- Mar 16: "Sprint 3D: Wire Browse by Category to article filter – mutual exclusivity with search, results grid display"
- Mar 16: "Sprint 3D: Wrap All Articles in collapsed expander by default – UX improvement to reduce cognitive overload"
- Mar 16: "Sprint 3D: Restore teal styling on All Articles expander using CSS :has() marker pattern"
- Mar 16: "Sprint 3D: Bold All Articles expander label to match section header visual weight"
- Mar 16: "Sprint 4A: Generate 5,000 synthetic assessments CSV – 200 patients, 3 tiers, validated distributions"
- Mar 16: "Sprint 4B: System Analytics Dashboard – page structure, data loading, KPI cards from synthetic CSV"
- Mar 16: "Sprint 4B: Update KPI cards to teal header style – consistent with platform design system"
- Mar 16: "Sprint 4B: KPI cards verified – teal headers rendering correctly with accurate values from synthetic CSV"
- Mar 16: "Sprint 4B: Add 4 Plotly charts – top symptoms, urgency donut, health authority donut, assessment trends"
- Mar 16: "Sprint 4B: Improve charts – clinical severity colours on symptoms bar, selective data labels on trends line"

- Mar 16: "Sprint 4B: Fix General symptoms bar colour from teal to blue to match legend annotation"
- Mar 16: "Sprint 4B: Add K-Means clustering – patient segmentation scatter plot and cluster summary cards"
- Mar 16: "Sprint 4B: K-Means clustering verified – 3 patient segments clearly separated by age, frequency and urgency"

4. Revised Timeline and Next Steps

With Sprints 3D, 4A, and 4B complete, the platform is near feature-complete. Only Sprint 5 (Deployment & Testing) and Sprint 6 (Documentation & Final Submission) remain. The timeline has been updated to reflect actual completion dates:

Sprint	Description	Dates	Status	Report
Sprint 0	Environment Setup	Jan 30–31	DONE	PR1
Sprint 1A–E	Foundation (Mockups, DB, Auth, Nav)	Feb 2–5	DONE	PR1
Sprint 2A–D	Symptom Assessment Chatbot (3-Layer Pipeline)	Feb 6–13	DONE	Midterm
Sprint 3A	Home Dashboard (UI rebuild from mockup)	Feb 25–27	DONE	PR2
Sprint 3B	Personal Health Dashboard + Enhancements	Feb 28 — Mar 8	DONE	PR2/PR3
Sprint 3E	Multi-Turn Chatbot Enhancement	Mar 8–9	DONE	PR3
Sprint 3C	BC Healthcare Facility Finder	Mar 9	DONE	PR3
Sprint 3D	Health Education Library	Mar 11–15	DONE	PR4
Sprint 4A	Synthetic Data Generation	Mar 16	DONE	PR4
Sprint 4B	System Analytics Dashboard	Mar 16	DONE	PR4
Sprint 5	Deployment + End-to-End Testing	Mar 17–20	In Progress	PR5
Sprint 6	Documentation + Final Submission	Mar 21 — Apr 10	Planned	PR5

4.1 What's Next: Sprint 5 and Sprint 6

Sprint 5 — Deployment and End-to-End Testing (Mar 17–20): Deploy the platform to Streamlit Cloud via the GitHub repository connection. Conduct end-to-end testing across all 6 pages, validating navigation flows, data persistence, chatbot multi-turn behaviour under edge cases, facility finder filter combinations, health library search, and system analytics date range filter. Identify and resolve any deployment-specific issues (e.g., file path resolution differences between local and cloud environments, API key configuration via Streamlit secrets).

Sprint 6 — Documentation and Final Submission (Mar 21 — Apr 10): Complete the Final Report documenting the full system architecture, design decisions, ML methodology, testing results, and reflections on AI tool usage. Prepare the Final Presentation demonstrating the live deployed platform. Update the GitHub README with deployment instructions and project overview.

4.2 Dual Machine Learning Architecture — Academic Framing

One of the major architectural milestones during this period was finalizing the platform's twofold machine learning system. Currently, the platform utilizes two separate machine learning approaches:

- **Supervised Learning (Random Forest):** For the Symptom Assessment chatbot, a Random Forest classifier is employed, trained on a labelled Kaggle dataset, to forecast diagnoses based on symptoms extracted. To maintain a high level of safety in the healthcare environment, the model uses a 70% confidence level as a performance criterion, and if it is lower than this, it refers to the OpenAI GPT-4o-mini an intentionally cautious design.
- **Unsupervised Learning (K-Means Clustering):** The System Analytics Dashboard applies K-Means clustering to discover natural patient segments from assessment behaviour data, without predefined labels. This reveals three clinically meaningful population groups that could inform targeted BC health interventions.

This combination of supervised prediction at a single patient level and unsupervised population analytics at a system level imitates real-world clinical decision support architectures, where both diagnostic tools and population health analytics are crucial elements of offering a full range of healthcare services.

4.3 Optional Feature: Community Pulse Dashboard (Time-Permitting)

If Sprint 5 deployment and testing complete ahead of schedule, a Community Pulse feature is planned as an additional portfolio enhancement. This would introduce a new page (7_🏠_Community_Pulse.py)

that extends the platform with a real-time community health intelligence capability. The planned feature would involve two phases:

- Phase 7 — Standalone Dashboard: Scrape healthcare discussions from Reddit (r/healthIT, r/medicine) using BeautifulSoup, score posts by clinical relevance and sentiment using OpenAI GPT-4o-mini with a healthcare-specific prompt, and present an interactive Streamlit dashboard with 4 KPI cards (Total Posts Analysed, % Positive, % Negative, Avg Importance Score), a Plotly sentiment bar chart, importance score histogram, Top 10 most important posts table with clickable Reddit URLs, and an analyst narrative section using st.info() callout boxes for key findings.
- Phase 8 — BC Health Platform Integration: Port the standalone Community Pulse page into the existing BC Health Platform, adapting the colour scheme to the platform teal (#1a6b5c) design system, adding a 'Refresh Data (Coming Soon)' placeholder button to signal future live-scraping capability, and redeploying to Streamlit Cloud.

This feature is explicitly out of core project scope and will only be pursued if all core sprints (3D, 4A, 4B, and Sprint 5 deployment) are fully completed and tested ahead of the final submission deadline.

It is mentioned here as a planned portfolio extension that showcases additional NLP pipeline capabilities web scraping, sentiment analysis, and community health intelligence apart from the core platform feature set. If there's no time for it, this feature will be mentioned in the Final Report as a future work suggestion.

5. AI Use Section

5.1 AI Tools Usage Summary

During this reporting period, AI tools were used to clarify technical concepts and understand framework-specific patterns before implementation. All architecture decisions — including the Health Library JSON schema design, the 3-tier patient engagement model, the K-Means feature selection, the clinical severity colour coding logic, and the BC Health Authority city mapping — were made independently based on clinical reasoning and BC health system knowledge. AI tools usage was more in-line with a typical scenario of consulting documentation or asking a classmate for a technical explanation. All the output was run through my own hands-on testing, bugs were determined through my own testing and fix strategies were also devised by me.

5.2 Table of AI Tools and Specific Use

AI Tool	Version	Specific Use	Value I Added
Claude (Anthropic)	Claude.ai	Clarifying JSON knowledge base design patterns for health information systems — understanding how to structure article fields (tags, source_url, banner_color) for both search filtering and UI rendering	Independently decided on the 8 article categories based on HealthLinkBC topic areas; designed the JSON schema myself including the source_url field to link to official HealthLinkBC pages, ensuring the platform always directs users to authoritative health information rather than relying on AI-generated content
Claude (Anthropic)	Claude.ai	Understanding CSS :has() selector patterns for styling Streamlit widgets — how to apply background colours to Streamlit containers without using HTML div wrappers that auto-close before widget rendering	Identified the ghost box bug through my own testing; designed the marker-based CSS pattern approach; all styling decisions (teal colour, border-radius, padding values) matched to the existing platform design system independently
Claude (Anthropic)	Claude.ai	Understanding synthetic data generation patterns — how to model realistic health	Independently designed the 3-tier patient model with clinically justified assessment frequencies; decided on the Emergency cap

AI Tool	Version	Specific Use	Value I Added
		assessment distributions with seasonal patterns and demographic stratification using numpy and pandas	threshold (5%) based on realistic emergency department utilization rates; chose the 20 symptom pool and severity classifications based on common BC health concerns
Claude (Anthropic)	Claude.ai	Clarifying K-Means clustering implementation in scikit-learn — StandardScaler normalization, KMeans parameters, and how to aggregate per-patient features from a transaction-level DataFrame	Independently selected the 4 clustering features (age, assessment count, urgent ratio, chronic ratio) based on clinical relevance; decided on n_clusters=3 to match the 3-tier patient model; designed the cluster naming logic using combined severity score; interpreted clinical meaning of each segment independently
Claude Code (VS Code)	Extension	Implementing Health Library page structure, search session state logic, CSS :has() marker injection pattern, and System Analytics Plotly chart configurations based on my detailed specifications	All page layout specifications, session state key names, clinical severity colour assignments, BC Health Authority city mappings, chart type selections, and K-Means feature engineering decisions were mine. All outputs tested across multiple scenarios; bugs identified through my own testing and resolved through my debugging approach

5.3 Statement of Originality

All architectural decisions for Sprint 3D, 4A, and 4B were independently made: the Health Library JSON schema and 8-category structure, the 3-tier synthetic patient engagement model with tier-specific clinical profiles, the clinical severity colour-coding framework for symptom visualization, the BC Health Authority city-to-region mapping, and the K-Means feature selection for patient segmentation. AI tools were used to understand how specific libraries and patterns work, not to generate architecture or design decisions. I applied AI tools to grasp the functioning of certain libraries and patterns rather than creating architecture or making design decisions. Claude Code assisted me in writing boiler template code from my detailed instructions. I carried out testing of all outputs, found errors through my own

testing and fixed bugs through my own debugging. The choice to use two different ML architectures (supervised Random Forest + unsupervised K-Means) was a thoughtful design decision in order to showcase two ML approaches in one healthcare platform.

Appendix A: AI Prompt History

The following documents AI tool interactions during this reporting period. AI was used to clarify technical concepts and understand framework-specific patterns. Each entry follows the Context / Prompt / Summary / Value I Added structure.

Prompt Session 1: JSON Knowledge Base Design for Health Library

Date: March 11, 2026 | **Tool:** Claude (Anthropic)

Context: Having identified 8 health topic categories for the library and mapped 18 article topics, I needed to clarify how to structure the JSON schema so that article fields could serve double duty — both as search filter targets and as UI rendering data — without redundant lookups. I also needed to understand how source_url fields should be framed given that the article content was AI-generated rather than directly extracted from HealthLinkBC.

My Prompt: *"I'm building a health information library backed by a JSON file. Each article needs to support: (1) keyword search matching against multiple fields, (2) category filtering, (3) expandable card rendering with full content, and (4) a Visit Source link to the official HealthLinkBC URL. What fields should the JSON schema include, and how should I frame AI-generated article summaries that link to official sources?"*

Summary of Response: Claude described a standard knowledge base schema with separate fields for search metadata (tags array), display content (summary vs full content), and source attribution (source_url). It explained that AI-generated summaries linking to authoritative sources is a legitimate design pattern used in health information platforms, where the AI content serves as a navigational summary layer directing users to verified sources rather than replacing them.

Value I Added: I independently decided on the specific 8 categories based on reviewing HealthLinkBC's own topic structure. I chose to include banner_color as a field to support the Popular Topics visual design — this was not suggested by Claude. I made the decision to require source_url for every article (not just popular ones) to ensure every article on the platform has a verifiable official source, reinforcing the platform's commitment to evidence-based BC health information.

Prompt Session 2: CSS :has() Selector Pattern for Streamlit Widget Styling

Date: March 13, 2026 | **Tool:** Claude (Anthropic)

Context: After identifying that Streamlit auto-closes raw HTML div tags before rendering widgets inside them (producing ghost boxes), I needed to understand an alternative CSS pattern that could style Streamlit's own generated DOM containers without injecting HTML wrappers around widget blocks.

My Prompt: *"In Streamlit, I cannot wrap widgets inside HTML div tags injected via st.markdown because the browser auto-closes the div immediately. I need to apply a teal background to a section containing multiple Streamlit widgets. Is there a CSS approach that targets Streamlit's own container elements from the outside rather than wrapping them from the inside?"*

Summary of Response: Claude explained the CSS :has() relational selector, which allows targeting a parent element based on whether it contains a specific child. By injecting a tiny invisible marker span (e.g.,) inside a Streamlit container, CSS can use div:has(.search-band-marker) to style that container from the outside without any HTML wrapper.

Value I Added: I identified the ghost box bug through my own testing and traced it to the HTML div auto-closing behaviour independently. The :has() pattern was new to me — Claude explained the concept — but I designed all the specific marker class names, CSS property values (background colours, border-radius, padding), and decided which containers needed the pattern applied. I also verified the fix across all affected sections and standardized the pattern for reuse across future sections.

Prompt Session 3: Synthetic Data Generation with Stratified Patient Tiers

Date: March 16, 2026 | **Tool:** Claude (Anthropic)

Context: Having designed the 3-tier patient model on paper, I needed to understand how to generate realistic stratified health assessment data in Python using numpy and pandas, specifically how to implement weighted random selection for symptom and urgency assignment at the per-patient level based on tier-specific profiles.

My Prompt: *"I'm generating a synthetic health assessment dataset with 3 patient tiers (Healthy/Occasional, Moderate, Chronic/High-Frequency), each with different assessment frequencies, age ranges, urgency distributions, and symptom profiles. In Python with numpy, how do I implement weighted random selection to assign urgency levels per-patient based on tier-specific probability arrays? How do I add a seasonal pattern (flu season spike for specific symptoms in Nov–Feb)?"*

Summary of Response: Claude explained numpy.random.choice with a p parameter for weighted selection, and the pattern of checking the assessment_date month to conditionally boost symptom weights during flu season months. It also described the post-generation validation pattern for enforcing global caps after per-record generation.

Value I Added: I independently designed the 3-tier model with clinically justified parameters: the specific age ranges per tier, the assessment frequency ranges, and the urgency weight distributions were all based on my reasoning about how different patient populations use health platforms. I chose the Emergency cap at 5% based on real BC emergency department utilization rates. I also designed the post-generation cap enforcement approach (downgrading excess Emergency records to Urgent) as a safety validation step. The 20-symptom pool was curated by me based on common BC health concerns covered by HealthLinkBC.

Prompt Session 4: K-Means Feature Engineering for Healthcare Patient Segmentation

Date: March 16, 2026 | **Tool:** Claude (Anthropic)

Context: Having decided to implement K-Means clustering for patient segmentation, I needed to understand how to aggregate transaction-level assessment records into per-patient feature vectors suitable for clustering, and how to apply StandardScaler normalization correctly before fitting KMeans.

My Prompt: *"I have a DataFrame with 5,000 assessment records where each row is one assessment for a patient. I want to cluster patients (not assessments) using K-Means. How do I aggregate to per-patient features (mean age, assessment count, urgent ratio, chronic ratio) from a transaction-level DataFrame using pandas groupby? And how should I apply StandardScaler — should I fit on the training data only, or fit_transform the full dataset for unsupervised clustering?"*

Summary of Response: Claude explained the pandas groupby + agg() pattern for per-patient feature aggregation, computing ratio fields as conditional counts divided by total counts. It clarified that for unsupervised clustering (no train/test split), fit_transform on the full dataset is appropriate since there is no risk of data leakage in the absence of a supervised target variable.

Value I Added: I independently selected the 4 clustering features based on clinical relevance: assessment frequency reflects engagement tier, urgent ratio captures clinical severity profile, and chronic ratio specifically captures the symptom pattern that distinguishes chronic patients from acute-illness patients. I decided on n_clusters=3 to match the 3-tier synthetic patient model, allowing direct comparison between the designed tiers and the discovered clusters. I designed the cluster auto-naming logic using combined chronic + urgent scores rather than single-feature ranking to produce more stable and clinically meaningful labels. The clinical interpretation of each cluster (and the BC health intervention recommendation) was my independent analysis.